03 – Model training (Phase 6)

Eight neural-network architectures were tuned with the same random search and compared with cross-validation inside the training rows. This notebook summarises every logged training run and **re-trains the chosen networks live** to show that the saved results are reproduced exactly.

In [1]:
```
# Setup: work from the project root so the project code and configs are found
import os, sys, json, platform, warnings
from pathlib import Path
ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
os.chdir(ROOT); sys.path.insert(0, str(ROOT))
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
from IPython.display import Image, Markdown, display
pd.set_option("display.precision", 3); pd.set_option("display.width", 160);
pd.set_option("display.max_columns", 20)
from src.utils.config import load_config, resolve_path
REPORTS = resolve_path(load_config("project")["paths"]["reports"])
FIG = resolve_path(load_config("project")["paths"]["figures"])
print("Project folder:", ROOT)
print("Python", platform.python_version(), "| numpy", np.__version__, "|
pandas", pd.__version__)
```

```
Project folder: C:\AL_outcome_predictor
Python 3.12.0 | numpy 2.5.4 | pandas 3.0.6
```

1. Training settings

In [2]:
```
mc = load_config("models")
print("Architectures:", ", ".join(mc["models"]))
print("Budget:", json.dumps(mc["budget"], indent=1))
print("Training:", mc["training"])
```

```
Architectures: mlp, reg_dnn, wide_deep, tabnet, mtl, entity_mlp, ae_mlp, resnet
Budget: {
 "feature_set_comparison": "default config x 3 feature sets x 15 folds",
 "random_search_configs": 20,
 "seeds": [
  0,
  1,
  2,
  3,
  4
 ],
 "search_seed": 2026,
 "learning_curve_fractions": [
  0.2,
  0.4,
  0.6,
  0.8,
  1.0
 ]
}
Training: {'optimizer': 'adam', 'max_epochs': 300, 'early_stopping_fraction': 0.1,
'restore_best_weights': True, 'torch_threads_per_worker': 1, 'n_jobs': 14}
```

2. Every training run is logged

`reports/phase06/runs.jsonl` holds one record per training run (model, settings, fold, seed and scores).

In [3]:
```
from src.models import cv
log = cv.read_log()
steps = pd.Series([r["step"] for r in
log]).value_counts().rename("runs").to_frame()
print("Logged training runs:", len(log))
steps
```

```
Logged training runs: 5590
```

Out[3]:

	runs
search	2400
stage2_search	2400
feature_sets	360
er4	240
final	40
learning_curve	40
single_task	40
check_ae	20
check_reg_dnn	15
check_wide_deep	15
check_entity	10
check_resnet	5
stage2_final	5

3. Re-run one cross-validation run from the random search

One logged search run (first configuration of the MLP, fold `r0f0`) is trained again from scratch and its scores are compared with the log.

In [4]:
```
cache = cv.build_fold_cache()
logged = next(r for r in log if r["step"] == "search" and r["model"] == "mlp"
and r["config_id"] == "c00" and r["split"] == "r0f0")
columns = json.loads((REPORTS / "phase06" /
"final_columns.json").read_text("utf-8"))
job = cv.make_job("search", "mlp", logged["cfg"], columns, "r0f0",
logged["seed"], config_id="c00")
again = cv.evaluate_run(job, cache["r0f0"])
pd.DataFrame({"logged in Phase 6": {t: logged["metrics"][t]["mae" if t ==
"zscore" else "macro_f1"] for t in logged["metrics"]},
              "re-run now": {t: again["metrics"][t]["mae" if t == "zscore" else
"macro_f1"] for t in again["metrics"]}})
```

Out[4]:

	logged in Phase 6	re-run now
zscore	0.226	0.226
zscore_band	0.633	0.633
university_group	0.267	0.267
degree_category	0.739	0.739

4. Comparing the eight architectures (cross-validation, after tuning)

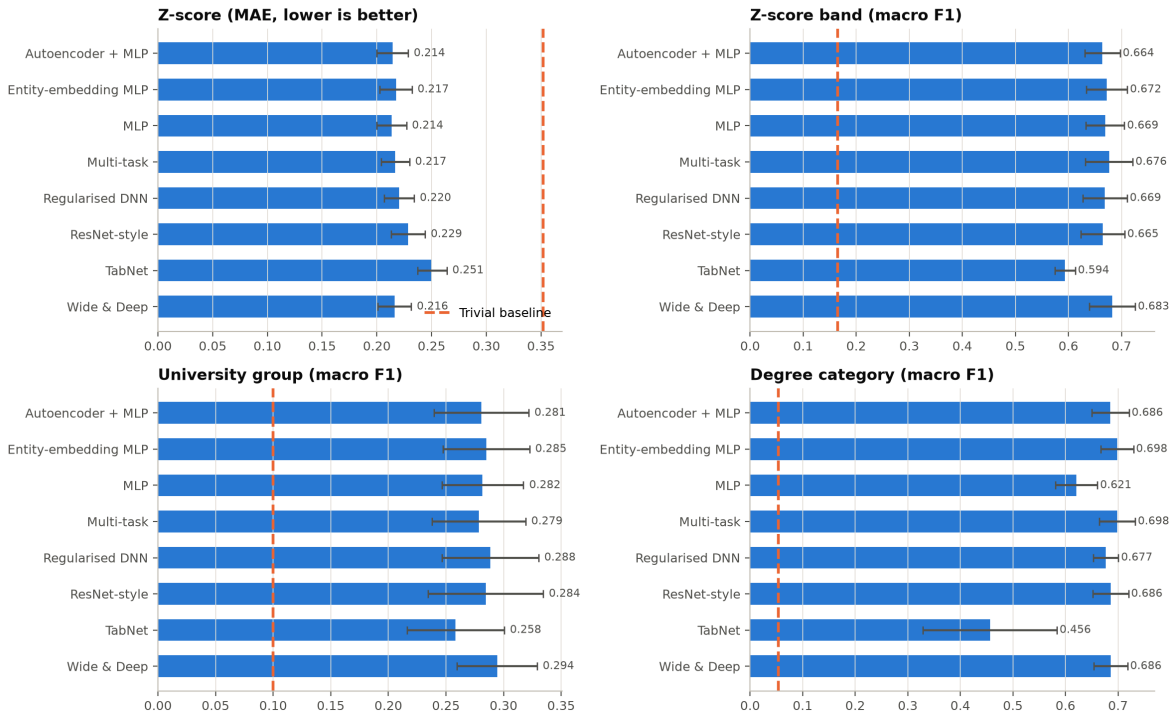
Lower mean rank is better. The locked rule picks the simplest network whose rank is within one standard deviation of the best; this chose the plain MLP.

In [5]:
```
ranks = pd.read_csv(REPORTS / "phase06" / "step3_architecture_ranks.csv")
display(ranks)
decisions = json.loads((REPORTS / "phase06" / "decisions.json").read_text("utf-8"))
print("Selected architecture:", decisions["architecture"]["selected"])
print("Its configuration:", decisions["configs"][decisions["architecture"]["selected"]])
display(Image(str(FIG / "phase06_cv_comparison.png"), width=760))
```

	model	mean_rank	sd_rank	n_params
0	wide_deep	3.350	1.242	2416.0
1	entity_mlp	3.742	0.941	6852.0
2	mtl	3.817	1.215	13012.0
3	ae_mlp	3.883	0.986	15954.0
4	reg_dnn	4.442	0.962	7696.0
5	mlp	4.575	0.735	1648.0
6	resnet	4.875	0.549	11856.0
7	tabnet	7.317	0.842	8004.0

```
Selected architecture: mlp
Its configuration: {'config_id': 'c12', 'cfg': {'lr': 0.000128, 'batch_size': 64, 'patience': 10, 'layers': 1, 'units': 32}}
```

Cross-validation inside the training split (15 folds, mean ± SD), tuned configurations



5. Train the chosen multi-output MLP live (seed 0)

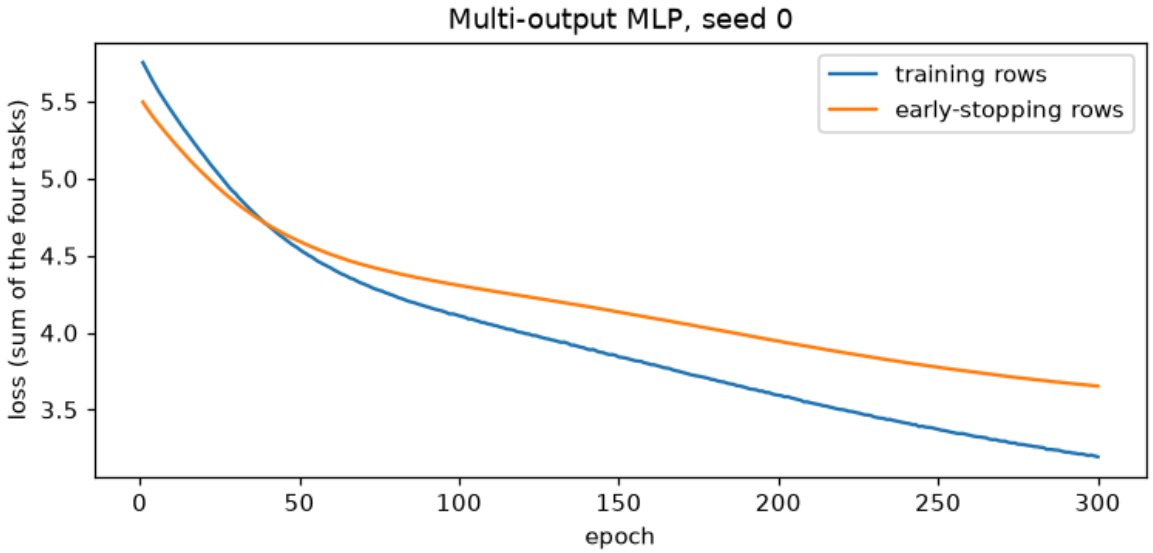
This is the training call used by the project (`src/models/train.py: fit`), on the training rows, with early stopping on a part of them. Nothing is saved.

In [6]:
```
import torch
from src.models.data import make_model_data, dims_for
from src.models.train import fit, predict
from src.models.run_phase06_final import context, records, load_model
configs, columns, selected, seeds = context()
entry = cache["trainval"]
crosses = mc["wide_crosses"]
train_data = make_model_data(entry["train_x"], entry["train_y"], selected,
columns, crosses)
val_data = make_model_data(entry["held_x"], entry["held_y"], selected, columns,
crosses)
model = fit(selected, configs[selected]["cfg"], train_data, ["zscore",
"zscore_band", "university_group", "degree_category"],
            0, dims_for(train_data, cv.N_DISTRICTS), mc["training"]
["max_epochs"], mc["training"]["early_stopping_fraction"])
print(model.network)
print(f"Parameters: {model.n_params} | epochs run: {model.epochs_run} | best
epoch: {model.best_epoch} | {model.seconds:.1f} s")
```

```
MLPNet(
  (body): Sequential(
    (0): Linear(in_features=34, out_features=32, bias=True)
    (1): ReLU()
  )
  (head): ModuleDict(
    (zscore): Linear(in_features=32, out_features=1, bias=True)
    (zscore_band): Linear(in_features=32, out_features=3, bias=True)
    (university_group): Linear(in_features=32, out_features=5, bias=True)
    (degree_category): Linear(in_features=32, out_features=7, bias=True)
  )
)
Parameters: 1648 | epochs run: 300 | best epoch: 300 | 3.2 s
```

Training curve of this run

In [7]:
```
import matplotlib.pyplot as plt
h = pd.DataFrame(model.history)
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(h["epoch"], h["train_loss"], label="training rows")
ax.plot(h["epoch"], h["es_loss"], label="early-stopping rows")
ax.set_xlabel("epoch"); ax.set_ylabel("loss (sum of the four tasks)");
ax.legend(); ax.set_title("Multi-output MLP, seed 0")
plt.tight_layout()
import io; buffer = io.BytesIO(); fig.savefig(buffer, format="png", dpi=110);
plt.close(fig)
display(Image(data=buffer.getvalue()))
```



The live model gives the same predictions as the saved model used by the app

In [8]:
```
saved = load_model("final", selected, "-", 0)
fresh_pred, saved_pred = predict(model, val_data), predict(saved, val_data)
for task in fresh_pred:
    print(f"{task:17s} largest difference on the 152 validation rows:
{np.abs(fresh_pred[task] - saved_pred[task]).max():.2e}")
```

```
zscore            largest difference on the 152 validation rows: 0.00e+00
zscore_band       largest difference on the 152 validation rows: 0.00e+00
university_group  largest difference on the 152 validation rows: 0.00e+00
degree_category   largest difference on the 152 validation rows: 0.00e+00
```

6. All five seeds re-trained, compared with the log (validation scores)

In [9]:
```
rows = []
for seed in seeds:
    job = cv.make_job("final", selected, configs[selected]["cfg"], columns,
"trainval", seed, config_id=configs[selected]["config_id"])
    new = cv.evaluate_run(job, entry)
    old = next(r for r in records("final", selected) if r["seed"] == seed)
    for task, metric in (("zscore", "mae"), ("zscore_band", "macro_f1"),
("university_group", "macro_f1"), ("degree_category", "macro_f1")):
        rows.append({"seed": seed, "task": task, "metric": metric, "logged":
old["metrics"][task][metric], "re-trained now": new["metrics"][task][metric]})
seeds_table = pd.DataFrame(rows)
seeds_table["same"] = np.isclose(seeds_table["logged"], seeds_table["re-trained
now"], atol=1e-9)
display(seeds_table)
print("All re-trained scores equal the logged scores:",
bool(seeds_table["same"].all()))
```

	seed	task	metric	logged	re-trained now	same
0	0	zscore	mae	0.211	0.211	True
1	0	zscore_band	macro_f1	0.649	0.649	True
2	0	university_group	macro_f1	0.281	0.281	True
3	0	degree_category	macro_f1	0.633	0.633	True
4	1	zscore	mae	0.213	0.213	True
5	1	zscore_band	macro_f1	0.682	0.682	True
6	1	university_group	macro_f1	0.270	0.270	True
7	1	degree_category	macro_f1	0.601	0.601	True
8	2	zscore	mae	0.214	0.214	True
9	2	zscore_band	macro_f1	0.650	0.650	True
10	2	university_group	macro_f1	0.332	0.332	True
11	2	degree_category	macro_f1	0.635	0.635	True
12	3	zscore	mae	0.220	0.220	True
13	3	zscore_band	macro_f1	0.669	0.669	True
14	3	university_group	macro_f1	0.346	0.346	True
15	3	degree_category	macro_f1	0.614	0.614	True
16	4	zscore	mae	0.216	0.216	True
17	4	zscore_band	macro_f1	0.681	0.681	True
18	4	university_group	macro_f1	0.355	0.355	True
19	4	degree_category	macro_f1	0.601	0.601	True

```
All re-trained scores equal the logged scores: True
```

7. Stage 2: the single-task Z-score MLP

The Z-score was predicted better by its own network (negative transfer in the multi-output model), as allowed by the locked rule. It is re-trained here in the same way.

In [10]:
```
s2 = decisions["stage2"]["zscore"]
print("Stage 2 network:", s2["selected"], s2["config_id"], s2["cfg"])
rows = []
for seed in seeds:
    job = cv.make_job("stage2_final", s2["selected"], s2["cfg"], columns,
"trainval", seed, config_id=s2["config_id"], variant="zscore", tasks=["zscore"])
    new = cv.evaluate_run(job, entry)
    old = next(r for r in log if r["step"] == "stage2_final" and r["seed"] ==
seed)
    rows.append({"seed": seed, "logged MAE": old["metrics"]["zscore"]["mae"],
"re-trained MAE": new["metrics"]["zscore"]["mae"]})
display(pd.DataFrame(rows))
pd.read_csv(REPORTS / "phase06" / "stage2_validation.csv")[["task",
"stage2_model", "stage2_validation", "multi_output_validation", "ci_low",
"ci_high"]]
```

```
Stage 2 network: mlp c12 {'lr': 0.000128, 'batch_size': 64, 'patience': 10,
'layers': 1, 'units': 32}
```

	seed	logged MAE	re-trained MAE
0	0	0.202	0.202
1	1	0.208	0.208
2	2	0.213	0.213
3	3	0.210	0.210
4	4	0.214	0.214

Out[10]:

	task	stage2_model	stage2_validation	multi_output_validation	ci_low	ci_high
0	zscore	mlp	0.209 ± 0.005	0.215 ± 0.004	0.002	0.018

8. Validation results of all eight networks (mean ± SD over 5 seeds)

All models are reported, including the ones not chosen. Baseline = always predicting the training average / most common class.

In [11]:
```
pd.read_csv(REPORTS / "phase06" / "validation_primary.csv")
```

Out[11]:

	model	n_params	seconds_mean	best_epoch_mean	zscore	zscore_baseline	zscore_band	zscore_b
0	ae_mlp	15954	14.846	77.4	0.210 ± 0.002	0.354	0.667 ± 0.005	
1	entity_mlp	6852	13.964	128.2	0.213 ± 0.001	0.354	0.674 ± 0.009	
2	mlp	1648	8.638	300.0	0.215 ± 0.004	0.354	0.666 ± 0.016	
3	mtl	13012	22.656	191.6	0.210 ± 0.003	0.354	0.654 ± 0.014	
4	reg_dnn	7696	2.504	66.4	0.212 ± 0.002	0.354	0.657 ± 0.016	
5	resnet	11856	2.804	73.2	0.217 ± 0.006	0.354	0.639 ± 0.019	
6	tabnet	8004	13.346	66.0	0.234 ± 0.007	0.354	0.638 ± 0.032	
7	wide_deep	2416	28.104	299.2	0.213 ± 0.002	0.354	0.663 ± 0.015	

In [12]:
```
display(Image(str(FIG / "phase06_learning_curves.png"), width=760))
```

Learning curves on the validation split (seed 0)

